



Fundamentos de Sistemas Embebidos

Facultad de Ingeniería

UNAM

Laboratorio Salas A y B

Profesor(a): Eduardo Fragoso Navarro

Asignatura: Fundamentos de Sistemas Embebidos

Grupo: 3

Número de Práctica(s): 4

Integrante(s): Tepal Briseño Hansel Yael - 320078282

Ugartechea González Luis Antonio - 320223822

Número de brigada: 10

Semestre: 9°

Fecha de entrega: 29 de septiembre del 2026

Observaciones: _____

CALIFICACIÓN: _____

Comunicación inalámbrica: RF transparente (HC-12)

Objetivo

Establecer una comunicación inalámbrica por radiofrecuencia (RF transparente) empleando dos módulos HC-12, con el fin de demostrar que el microcontrolador XIAO ESP32-S3 puede transmitir y recibir datos de manera confiable empleando exactamente el mismo código utilizado en la práctica de comunicación alámbrica UART.

Resultados

Ejercicio 1

Se llevó a cabo la implementación física del circuito esquemático sobre la protoboard. En la siguiente imagen se presenta el armado final, donde se ve la integración de la tarjeta de desarrollo XIAO ESP32-S3 conectada a uno de los módulos HC-12, y el otro módulo conectado a un convertidor TTL-USB a UART para la comunicación con la computadora. Se puede observar la conexión del módulo HC-12 al microcontrolador, así como el cableado del convertidor USB a UART hacia la computadora.

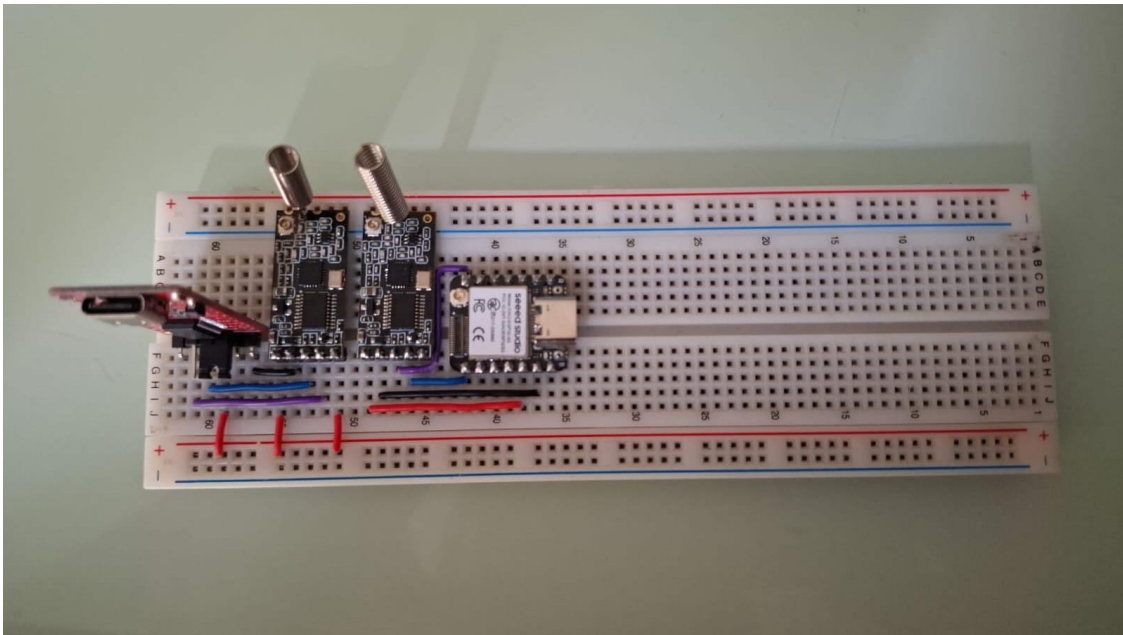


Figure 1: Circuito final implementado en la protoboard.

Ejercicio 2

Para el segundo ejercicio, se descargó el código fuente desde el proyecto ej_uart el cual se encarga de establecer la comunicación serial. Es importante destacar que no fue necesario modificar ni una sola línea de código respecto al ejercicio de comunicación alámbrica, debido a que el enlace de radiofrecuencia con los módulos HC-12 opera de manera transparente.

La comunicación se lleva a cabo mediante la configuración del periférico UART de la ESP32-S3[cite: 7]. Primero, se inicializan los parámetros de comunicación a 9600 baudios, 8 bits de datos, sin paridad y un bit de parada[cite: 7]. Los pines asignados para esta tarea son el GPIO43 para transmisión (TX) y el GPIO44 para recepción (RX)[cite: 7].

```
1 void uart_configurar(void)
2 {
3     uart_config_t configuracion = {
4         .baud_rate = UART_BAUD,
5         .data_bits = UART_DATA_8_BITS,
6         .parity     = UART_PARITY_DISABLE,
7         .stop_bits  = UART_STOP_BITS_1,
8         .flow_ctrl  = UART_HW_FLOWCTRL_DISABLE,
9     };
10    uart_param_config(UART_PUERTO, &configuracion);
11    uart_set_pin(UART_PUERTO, PIN_TX, PIN_RX,
12                UART_PIN_NO_CHANGE, UART_PIN_NO_CHANGE);
13    uart_driver_install(UART_PUERTO, 256, 256, 0, NULL, 0);
14 }
```

Listing 1: Configuración del puerto UART

En cuanto a la transmisión, el microcontrolador calcula periódicamente un valor correspondiente a una onda senoidal, escalado entre 0 y 255. Este byte se envía a través del puerto utilizando la función `uart_write_bytes`. A esta función se le pasa la dirección de memoria de la variable (`&valor_enviado`) dado que está diseñada de forma genérica para mandar bloques de datos mediante punteros.

```
1 // ejemplo 1: onda senoidal, un byte (0-255)
2 uint8_t valor_enviado = (uint8_t)(127 + 127 * sin(angulo));
3 uart_write_bytes(UART_PUERTO, &valor_enviado, 1);
4 angulo += 0.2;
5 if (angulo > DOS_PI) {
6     angulo = 0;
7 }
```

Listing 2: Transmisión de datos (Tx)

Simultáneamente, la ESP32-S3 se mantiene a la escucha de datos entrantes en el bus serial. Mediante la función `uart_read_bytes`, se lee el buffer de recepción y la función emplea el puntero `&valor_recibido` para escribir el dato entrante directamente en esa dirección de memoria. Si

el programa detecta que el byte recibido equivale a 128 (VALOR_PRENDE), el LED integrado de la tarjeta XIAO se enciende; con cualquier otro valor, se mantiene apagado. Este proceso se repite en un ciclo infinito con retardos de 500 milisegundos[cite: 7].

```
1 // recibir: prender o apagar el LED
2 uint8_t valor_recibido;
3 int bytes_leidos =
4     uart_read_bytes(UART_PUERTO, &valor_recibido, 1, 0);
5 if (bytes_leidos > 0) {
6     if (valor_recibido == VALOR_PRENDE) {
7         gpio_set_level(PIN_LED, LED_PRENDIDO);
8     } else {
9         gpio_set_level(PIN_LED, LED_APAGADO);
10    }
11 }
12 vTaskDelay(pdMS_TO_TICKS(500)); // periodo: 500 ms
```

Listing 3: Recepción de datos y control de actuador (Rx)

Ejercicio 3

Antes de validar el funcionamiento, es importante tener en cuenta que en sistemas operativos como linux, los archivos de dispositivos seriales (como /dev/ttyACM0 o /dev/ttyUSB0 utilizados por el convertidor TTL-USB o la tarjeta ESP32) son propiedad de el usuario root y de un grupo llamado dialout. Por lo tanto, para no tener problemas de permisos al abrir el puerto serial, es necesario agregar a nuestro usuario al grupo dialout mediante el comando `sudo usermod -a -G dialout $USER` y posteriormente cerrar sesión y volver a iniciar. Con este cambio, la aplicación SerialTest puede abrir el puerto serial sin que ocurra algun error en el acceso.

Para comprobar el funcionamiento del sistema, se instalo el software de SerialTest para monitorear la comunicación entre la computadora y el módulo HC-12 conectado a la ESP32-S3. Para ello, desde la aplicación SerialTest, se seleccionó el puerto /dev/ttyUSB0 correspondiente al convertidor USB a UART y se seleccionó la opción de open para configurar la comunicación a 9600 baudios, 8 bits de datos, sin paridad y un bit de parada.

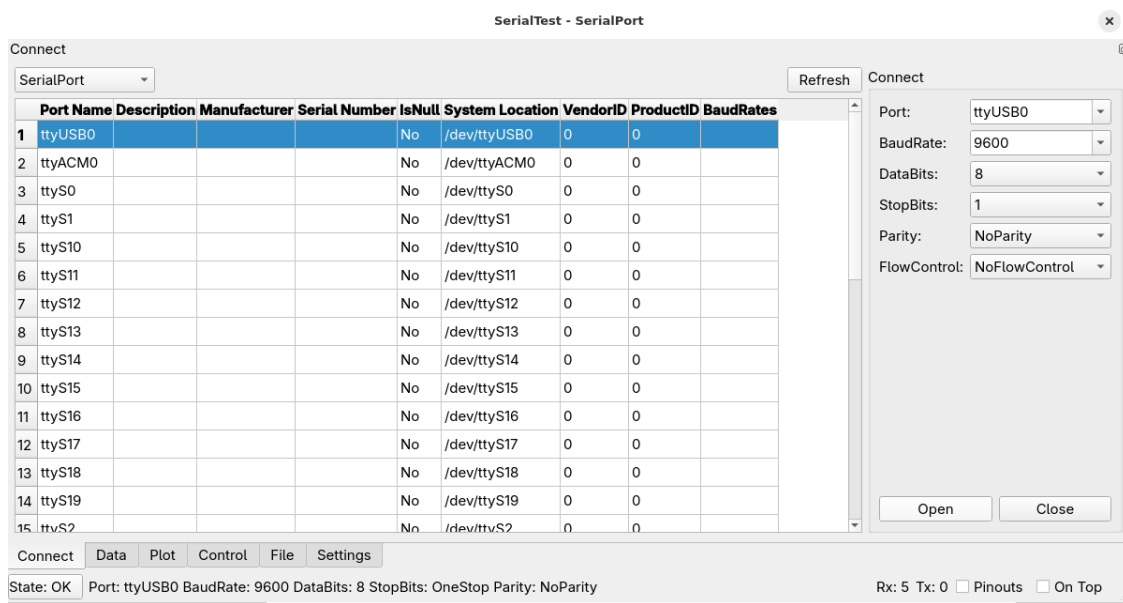


Figure 2: Configuración de la comunicación serial en la aplicación SerialTest.

Una vez configurado el puerto serial, se abre la ventana de DATA para visualizar los datos que se están recibiendo desde la ESP32-S3. En la siguiente imagen se puede observar que el valor recibido es una onda senoidal que oscila entre 0 y 255, tal como se esperaba. Además, al enviar un valor de 128 desde la ESP32-S3, el LED integrado de la tarjeta XIAO se enciende, confirmando que la comunicación bidireccional funciona correctamente.

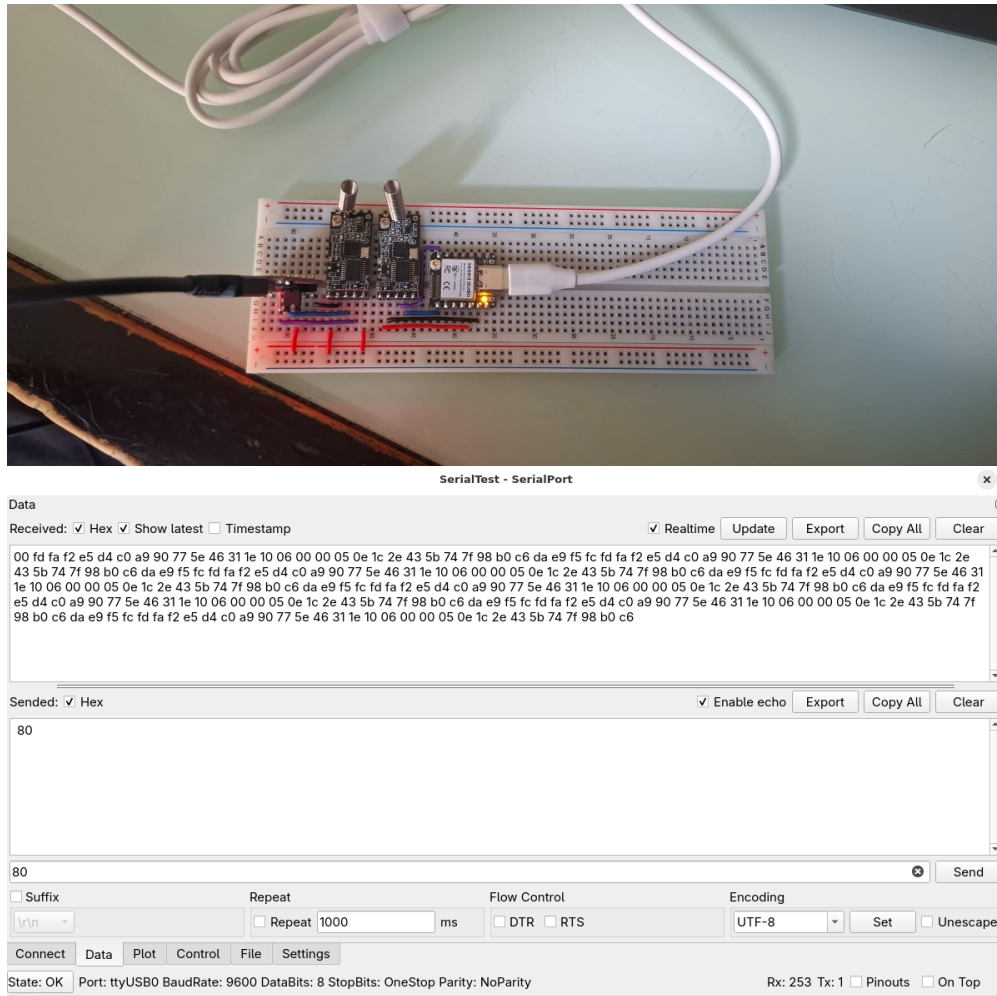


Figure 3: Verificación del encendido del LED integrado de la tarjeta XIAO ESP32-S3 al recibir el valor 128 desde la computadora.

Como puede observarse en la captura de pantalla de la aplicación SerialTest, la comunicación inalámbrica mediante los módulos HC-12 funciona correctamente, permitiendo la transmisión y recepción de datos entre la computadora y la tarjeta XIAO ESP32-S3. Ya que al enviar el valor de 128 (80 en hexadecimal) desde la ESP32-S3, el LED integrado de la tarjeta se enciende. Por otra parte, al enviar cualquier otro valor, el LED se mantiene apagado, confirmando que la comunicación bidireccional es estable y confiable.

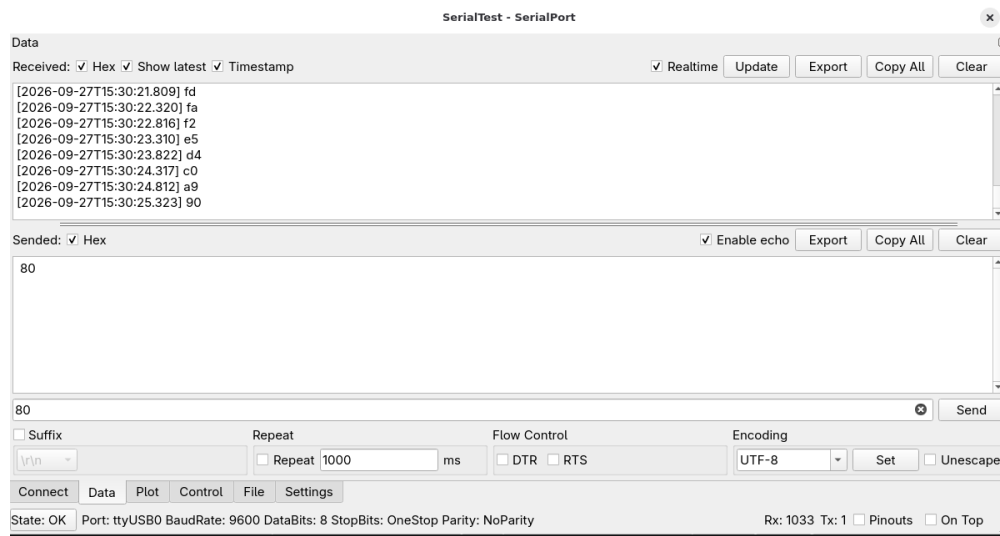


Figure 5: Verificación de la periodicidad de la recepción de datos cada 500 ms.

Finalmente, para probar el alcance de la comunicación inalámbrica, se realizó una prueba de distancia entre los dos HC-12. Para ello, se colocaron en diferentes habitaciones de la casa.

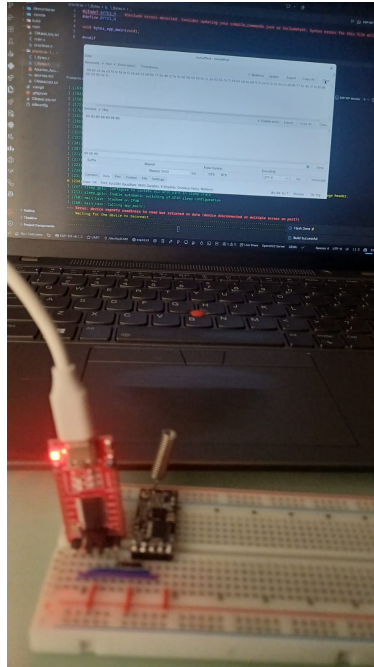


Figure 6: Prueba de alcance de la comunicación inalámbrica entre los módulos HC-12 en diferentes habitaciones.

Como puede verse a pesar de encontrarse alejadas un aproximado de 8 metros y con paredes de por medio, la comunicación entre los módulos HC-12 se mantiene estable y confiable, pues al enviar el valor de encendido (128) el cambio resultaba inmediato en el LED integrado de la tarjeta XIAO ESP32-S3, confirmando que la comunicación bidireccional funciona correctamente incluso a esa distancia y con obstáculos.

Análisis

Como se documentó en el Ejercicio 2, el aspecto más destacable de esta implementación es que no se tuvo que modificar ni una sola línea del código fuente de la ESP32-S3 en comparación con la implementación alámbrica. Esto se justifica debido a que los módulos HC-12 operan en lo que se conoce como modo transparente. Para el microcontrolador, conectarse a los pines RX y TX del HC-12 es completamente indistinguible de estar conectado a un cable UART tradicional; toda la modulación, empaquetado y transmisión de radiofrecuencia es gestionada internamente por el hardware del módulo.

En las pruebas físicas, se comprobó que la comunicación seguía siendo confiable incluso a una distancia aproximada de 8 metros y con paredes de por medio. Esta fiabilidad frente a obstáculos es característica de los transceptores sub-GHz (banda de 433 MHz típica de los HC-12), cuyas ondas electromagnéticas tienen una gran capacidad de penetración en interiores, superando en este aspecto a tecnologías de mayor frecuencia como el Wi-Fi.

Para que este enlace funcione, es un requisito absoluto que ambos módulos se encuentren en el mismo canal de frecuencia y velocidad. Si los HC-12 quedaran configurados en canales distintos, la comunicación fallaría por completo, ya que un módulo estaría transmitiendo en una frecuencia que el otro no está escuchando. Afortunadamente, estos módulos vienen configurados de fábrica para operar en el mismo canal predeterminado. Si en un entorno con interferencias se necesitara cambiar el canal, bastaría con poner en estado bajo (GND) el pin SET, lo que saca al módulo del modo transparente y lo entra al modo de comandos AT. Una vez en este modo, se envía el comando AT+Cxxx (donde xxx es el número del canal, por ejemplo, AT+C005) para reconfigurarlo.

Conclusiones

La transición de una comunicación alámbrica a una inalámbrica presenta ventajas evidentes, siendo la principal la eliminación de las barreras físicas. Esto permite una total movilidad del prototipo y facilita el aislamiento eléctrico entre la etapa de control (PC) y el sistema integrado (microcontrolador). Sin embargo, también conlleva desventajas frente al UART tradicional: el medio inalámbrico es susceptible a interferencias electromagnéticas externas, el alcance está limitado por el entorno físico, y se requiere circuitería y alimentación adicional en el extremo remoto para hacer funcionar el módulo de radio.

A nivel de seguridad, la comunicación implementada presenta una vulnerabilidad crítica: los datos viajan "en texto claro" a través de un medio público y compartido. Dado que el sistema solo verifica si el valor recibido es 128 para encender el actuador (LED)[cite: 12, 13], cualquier persona dentro del rango de cobertura que tenga un módulo en el mismo canal podría realizar un ataque de inyección de datos para encender o apagar el dispositivo sin autorización, o bien, interceptar pasivamente la información que transmite el microcontrolador.

Para reducir estos riesgos sin perder la utilidad del enlace de RF transparente, es imperativo

implementar seguridad a nivel de software. Esto se lograría aplicando algoritmos de cifrado a los bytes antes de mandarlos por la función `uart_write_bytes`[cite: 12, 13] y descifrándolos al recibirlos, o bien, diseñando un protocolo de tramas que incluya una dirección lógica del destinatario y sumas de verificación (checksums) para descartar comandos de emisores no válidos.

NOTA: Si se desea consultar el código fuente completo del proyecto, se puede acceder al repositorio de GitHub en la siguiente dirección: https://github.com/lethalSopape/r/laboratorio_sistemas_embebidos